What we tried that did not work: the harmful-twin line and its follow-ups

Summary written 2026-10-25. It covers 2026-10-18 to 2026-10-23. Every number below is taken from a result file in the repo or on D:, listed under each item. The longer technical record is `reports/harm_twin_negative_results.md`.

The goal in one paragraph

We want a **certificate**: a statement, true with 90% confidence, that a model's **breach rate** (how often its answer is flagged harmful) is below some bound. The plain way is to sample prompts at random, label the answers, and put an exact binomial bound (Clopper-Pearson) on the count. The idea was to train a **twin**, a second model that knows where the main model tends to fail, and use it to choose *which* samples to label, so the same number of labels gives a tighter bound. Every attempt to make the twin pay for itself failed. Plain random sampling with the exact bound won each time, on the average breach rate.

At a glance

#	What we tried	What we expected	What happened
1	Twin that writes **responses** (E1)	Twin's harmful answers resemble the main model's, so reweighting is cheap	Twin wrote short, odd answers the main model would never write; it was a far **worse** guide than the main model itself
2	Twin warm-started by imitation instead of RL (E1 check)	Fixes 1 if RL was the cause	Same failure (184 vs 187); RL was not the cause
3	Reweighting maths on a toy, with a perfect twin (pre-E5)	Bound at most half as wide as plain sampling	**2.05x wider** even with a perfect twin
4	Twin that picks **prompts** (E6)	Bound at most half as wide as plain sampling	1.35x to 6.0x **wider**; twin no better than picking prompts at random
5	Better ways to group prompts (zero-GPU check on E6 data)	Some grouping concentrates the harm	Best usable grouping explained 5% of the variance; grouping made the bound up to 7x wider
6	"Wrong model?" Group by **attack family** on a different model (Stage A)	Real attack structure makes grouping pay	Grouping made the bound 6.5x to 28.6x wider
7	Most favourable split, chosen with hindsight (Stage A, post hoc)	At least this one wins	Still 1.27x to 1.33x wider
8	Full E2-E5 programme (alternating twin and defender)	Run after E1	Never run: GPU time made it infeasible, then E1's failure removed its premise

#	What we tried	What we expected	What happened
9	First adaptive rule for the per-family guarantee (rule A)	Saves labels at any stopping point	Saved labels only if you finish; certified **nothing** until about 1,500 labels

Items 1–7 are the main negatives. Items 8 and 9 are a blocked plan and a design flaw (since fixed). The two things that *did* work are at the end.

1. The response-space twin (E1)

Tried: train the twin with reinforcement learning to produce the harmful answers the main model (Qwen3-1.7B) produces, then estimate the breach rate from the twin's answers, reweighted back to the main model.

Result: the reweighting only works if the twin's harmful answers look like the main model's. They did not. The gap between the two (forward KL) had a median of **187 nats** over the 41 prompts we could measure (range 131–232). The main model itself, used as the guide, has a gap of about **2 nats**. In registered terms: 1 pass of 4 (E1b, harmful fraction 0.85, passed; the twin's answers were **0.21x** the main model's length; **0 of 41** prompts had a gap below 1 nat).

Why: RL pushes a model toward the *one* kind of output that scores best (mode-seeking). The twin collapsed onto short harmful answers. The main model's real harmful answers are long, so the twin gives them almost zero probability, and the reweighting breaks on exactly the answers that matter.

Sources: `results/twin/e1/report.md`, `/mnt/d/seldonian-runs/twin/e1/forward_kl.json`.

2. Imitation instead of RL (E1 follow-up)

Tried: a twin trained by plain imitation of harmful answers (supervised warm start), to see whether RL was the problem.

Result: median gap **184 nats** (imitation) vs **187** (RL). Per prompt, nearly identical. The training method was not the cause; the twin's whole approach in response space was.

Source: `/mnt/d/seldonian-runs/twin/e1/forward_kl_warm_vs_rl.json`.

3. The reweighting maths, even with a perfect twin (pre-E5 toy)

Tried: before spending GPU time, an exact toy (20 prompts, 400 enumerable answers, breach rate 0.021) with a *perfect* twin, to check that the bound gets tighter.

Result: the estimate's spread did fall a lot (to 0.12–0.19 of plain sampling's), but the **bound** at 300 labels was **2.05x wider** than plain sampling's, against a target of at most 0.5x. It only overtook plain sampling at about 3,000 labels.

Why: to stay valid whatever the twin does, the reweighted bound mixes in plain sampling (a fraction alpha) and must allow for weights as large as 1/alpha. That allowance costs more than the spread it saves. The exact binomial bound is very tight on a rare yes/no event and pays no such cost. This was the first sign that the problem is in the **bound**, not the twin.

Source: `results/twin/e5_toy/report.md`.

4. The prompt-space twin (E6)

Tried: move the twin from answers to **prompts**: a model that scores how likely each prompt is to cause a breach, used to choose which prompts to sample. Real data: 1,200 prompts, 32 WildGuard-labelled answers each (10 GPU-hours).

Result: 1 pass of 4. The bound stayed valid (coverage 0.99–1.00). But at the main setting the twin's bound was **0.0308 wide vs 0.0126** for plain sampling (2.4x wider; 1.35x to 6.0x across all 12 settings). The twin's ranking of prompts was weak (rank correlation 0.30, target 0.40). Most telling, the twin was no better than a word-count baseline or than **picking prompts uniformly at random** (0.0308 vs 0.0312 vs 0.0314).

Why: the same 1/alpha cost as item 3, plus a twin that could not find the risky prompts.

Source: `results/twin/e6/report.md`.

5. Better groupings of the same prompts (zero-GPU check)

Tried: split the breach rate's variance into the part due to *which prompt* was drawn and the part due to the randomness of the answer, then see how much of the first part any grouping of prompts captures.

Result:

- Half the variance (0.50) is answer-to-answer randomness on the same prompt. No choice of prompts can remove it, so at most a 2x gain was ever available.
- Share of variance captured: adversarial/benign 0.007, source 0.007, keyword count 0.000, prompt length 0.000, clusters of the model's own features 0.002–0.016, the twin's score **0.053**. Only knowing the true per-prompt rates (impossible in practice) reached 0.197.
- Grouping by the twin's score made the 300-label bound **7x wider** (0.1245 vs 0.0169).
- The data already on disk (38,400 labels) gives a bound about 18x tighter (0.00096 vs 0.0169) than plain sampling with 300 labels. On a fixed pool, labelling everything wins.
- Making harm 20x rarer (rate 0.001) helped plain sampling more than grouping (0.0036 vs 0.0121 at 500 labels).

Source: the E6 gate output (session record); summarised in `reports/harm_twin_negative_results.md`.

6. Was it the model? Grouping by attack family (Stage A)

Tried: a different model (Granite-3.3-2B) and prompts with real structure: 1,000 harmful prompts in 7 attack families (plain and 6 encodings: leetspeak, reverse, Caesar, ROT13, Atbash, base64), 24 labels each. Registered before looking.

Result: NO-GO.

- Only leetspeak breached (8.2%); plain was 0.9%, the other five encodings 0%. The model answers encoded prompts but mostly cannot decode them.
- Family explained **4.3%** of the variance (needed 10%).
- Certifying by family made the bound **6.5x** (simple split of labels) to **28.6x** (best-informed split) wider than plain sampling.

- Even on the "did it refuse?" label, where family explains 23.5% of the variance (more than any grouping in item 5), grouping was 4.5x wider.

Why: each family needs its own bound, at a seventh of the error budget. A family that never breaches still costs many labels to bound near zero, and five families did.

Source: `results/twin/stage_a/report.md`.

7. The most favourable split, with hindsight (Stage A, post hoc)

Tried: two groups, leetspeak vs everything else, chosen *after* seeing the rates, with every possible split of the 300 labels.

Result: best case still **1.27x to 1.33x wider** than plain sampling. Not registered; it does not change the verdict, it confirms it.

Source: `results/twin/stage_a/report.md` (post-hoc section).

8. The full alternating programme (E2-E5), never run

The original plan alternated the twin with a defender being trained against it (E2-E5). On the 12 GB GPU, generation plus the 4B judge ran at about 0.77 s per labelled answer. E1 as first registered needed 85.6 GPU-hours against an 8-hour budget, and E3 was about 14x over its 30 hours, so the plan was trimmed by amendment. E1 then refuted the twin as a sampling guide, which removed the premise for E2-E5. None was run.

Source: `.planning/harm-twin/HANDOFF.md` (2026-10-20 section), registration amendments.

9. The first per-family rule stopped short (rule A)

Context: the per-family guarantee (below) worked. But its labelling rule, which spreads labels evenly over the families still open, had a flaw found in a planning check: every family finishes at about the same moment.

Result: stopping at 300, 600 or 1,000 labels, rule A had certified **0 of 7** families, while plain sampling had certified 0.7, 1.1 and 3.7. For volunteers who may stop early, that is useless. It also needed 1.94x more labels than plain sampling at a 2% threshold.

Fixed by rule F (finish one family before starting the next): 1.0, 2.8 and 4.6 families at the same stopping points, same total cost.

Sources: `results/per_family/report.md`, `results/per_family_f/report.md`.

The common thread

1. **Rare events favour the simple bound.** At breach rates of 1–2%, the exact binomial bound on plain random samples is already tight. Every alternative paid a fixed cost that the rare event could not repay: the 1/alpha allowance in items 3–4, and the cost of bounding near-zero groups in items 5–7.
2. **Half the variance cannot be touched.** The answer on a single prompt is itself random. Even perfect prompt selection could gain at most 2x.
3. **The cost was in the certificate, not the sampler.** A better twin could not help, because even a *perfect* one lost (item 3).

4. **Labels were cheap, so there was nothing to save.** Judge labels cost about 0.2 s each. A smart sampler only has a job when labels are scarce, for example human labels.
5. **The labels are the judge's, not the truth.** On your 225 hand labels the judges' false-alarm rate was 18–23%, and answers to benign prompts were flagged at least as often as answers to plain harmful ones (1.36% vs 0.90% in Stage A). Every result above certifies the judge's rate.

What worked (for balance)

- **Validity never failed.** Every bound held its 90% confidence wherever it was measured, including with deliberately broken twins.
- **A per-family guarantee (registered GO, 2026-10-23).** Certifying *each* attack family, instead of the average, is where structure helps. Choosing which family to label next certified all 7 with **2,441** labels against **7,000** for uniform sampling. It also showed that the average hides problems: an average certificate passes "below 5%" in 99.7% of runs while leetspeak breaches at 8.2%. Rule F (item 9) keeps this gain when labelling stops early. Sources: `results/per_family/`, `results/per_family_f/`.

Not yet tried

- The per-family design with **human labels** (planned: you plus friends, rule F).
- A learned allocator on a pool where the groups are *not* known in advance.
- A single joint bound across groups instead of one bound per group.